Parentheses / Bracket Validation

20. Valid Parentheses

Solved

Easy | Topics | Companies | Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

if a opening bracket push a corresponding closing bracket

if a closing bracket -> if equal to top , you pop

else if stack is empty or not match you return false

after everything return true

1249. Minimum Remove to Make Valid Parentheses

Solved

Medium | Topics | Companies | Hint

Given a string s of `'('` , `')'` and lowercase English characters.

Your task is to remove the minimum number of parentheses (`'('` or `')'`, in any positions) so that the resulting *parentheses string* is valid and return **any** valid string.

Formally, a *parentheses string* is valid if and only if:

- It is the empty string, contains only lowercase characters, or
- It can be written as `AB` (`A` concatenated with `B`), where `A` and `B` are valid strings, or
- It can be written as `(A)`, where `A` is a valid string.

Your solution is already the standard **O(n)** stack solution. I'd only improve the comments, naming, and make the intent a little clearer.

```cpp
class Solution {
public:
    // TC = O(n)
    // SC = O(n)

    string minRemoveToMakeValid(string s) {

        // Stores indices of unmatched '('
        stack<int> st;

        // Mark invalid parentheses with a placeholder ('!')
        // and remove them in one final pass.
        for (int i = 0; i < s.size(); i++) {

            if (s[i] == '(') {
                st.push(i);
            }

            else if (s[i] == ')') {

                // No matching '(' exists.
                if (st.empty()) {
                    s[i] = '!';
                }
                // Match this ')' with the latest '('.
                else {
                    st.pop();
                }
            }
        }

        // Any '(' left in the stack never found a matching ')'.
        while (!st.empty()) {
            s[st.top()] = '!';
            st.pop();
        }

        // Build the final string by skipping invalid characters.
        string ans;
        for (char ch : s) {
            if (ch != '!')
                ans += ch;
        }

        return ans;
    }
};
```

Interview intuition

```cpp
// Idea:
//
// 1. Push every '(' index.
// 2. When ')' is found:
//      - If a matching '(' exists, pair them.
//      - Otherwise mark ')' for removal.
// 3. Any '(' left in the stack are unmatched, so remove them.
// 4. Build the answer by skipping all marked characters.
```

Small improvements over your code

- Removed the commented-out `// int ans = 0;`.
- Declared `ans` only when it's actually needed.
- Added comments explaining **why** each step is performed rather than **what** the code does.
- Used `char ch` instead of `auto it` for better readability.

This is exactly the kind of clean, interview-ready code that interviewers like to see.

1021. Remove Outermost Parentheses

Solved

Easy | Topics | Companies | Hint

A valid parentheses string is either empty `""`, `"(" + A + ")"`, or `A + B`, where `A` and `B` are valid parentheses strings, and `+` represents string concatenation.

- For example, `""`, `"()"`, `"(())()"`, and `"(()(()))"` are all valid parentheses strings.

A valid parentheses string `s` is primitive if it is nonempty, and there does not exist a way to split it into `s = A + B`, with `A` and `B` nonempty valid parentheses strings.

Given a valid parentheses string `s`, consider its primitive decomposition: `s = P1 + P2 + ... + Pk`, where `Pi` are primitive valid parentheses strings.

Return `s` *after removing the outermost parentheses of every primitive string in the primitive decomposition of* `s`.

Example 1:

C++ | Auto

```cpp
class Solution {
public:
    string removeOuterParentheses(string s) {
        stack<char> st;
        string temp ="" , ans="";

        for(auto &ch: s){
            if(ch=='('){
                st.push(ch);
                temp+=ch;
            }
            else{
                st.pop();
                temp+=ch;
                if(st.empty()==1){
                    ans+=temp.substr(1, temp.length()-2);
                    temp="";
                }

            }
        }
        return ans;
    }
```